

Design and Evaluation of a RAG Chatbot for Pasig City Disaster Preparedness

Justine Bradley Tulio
Bachelor of Science in Information
Technology
Jose Rizal University
Manila, Philippines
justinebradley.tulio@my.jru.edu

Abstract—Access to clear and relevant information is important during disasters, especially when people need guidance on flooding, earthquakes, fires, evacuation, and emergency contacts. This study developed a domain-specific Retrieval-Augmented Generation (RAG) chatbot using disaster preparedness information for Pasig City. The system loads and chunks custom text documents, generates embeddings using `all-MiniLM-L6-v2`, stores them in ChromaDB, and retrieves relevant information before generating an answer through a Groq-hosted language model. The baseline configuration used a temperature of 0 with an explicit fallback rule for unsupported questions. A hallucination stress-test increased the temperature to 1.0 and removed the fallback rule. The baseline correctly rejected an out-of-domain question, while the modified configuration produced information unsupported by the retrieved documents. These results demonstrate how retrieval and prompt restrictions can work together to improve grounding in a domain-specific chatbot.

Keywords—Retrieval-Augmented Generation, disaster preparedness, RAG, ChromaDB, hallucination, Pasig City

I. SYSTEM ARCHITECTURE

The RAG chatbot was developed for the domain **Disaster Preparedness and Emergency Information for Pasig City**. The main problem addressed by the project is the need for quick access to preparedness procedures, emergency contacts, and evacuation information from a small collection of domain-specific documents. Instead of allowing the language model to depend entirely on its pre-trained knowledge, the system first retrieves information that is relevant to the user's question and provides that information to the model as context.

A. Dataset and Information Sources

The dataset was compiled from multiple sources related to disaster preparedness and emergency information rather than relying on a single document. The information was organized into three text files covering: (1) fire and earthquake preparedness, (2) flood and typhoon preparedness, and (3) barangay emergency hotlines and evacuation centers.

The flood preparedness material was obtained from Pasig City Government information and includes actions before,

during, and after flooding. The same dataset also contains emergency go-bag recommendations from a separate social media publication. The flood guidance includes preparing supplies, knowing evacuation routes, avoiding flooded areas, following evacuation instructions, and waiting for authorities before returning to affected areas.

The fire and earthquake file also combines information from different relevant sources. The earthquake procedure is based on DOST-PHIVOLCS guidance on the “Drop, Cover, and Hold” method, while the fire-prevention and fire-safety information identifies Pasig City Government and BFP-Pasig materials as its sources. The file additionally contains guidance for cooking-oil fires and the PASS method for operating a fire extinguisher.

The third file contains barangay emergency contact numbers published through the Pasig City Government and evacuation-center information identified from Pasig Public Information Office material. Bringing these different sources together gave the chatbot a small but varied knowledge base containing preparedness actions, emergency contacts, and evacuation information relevant to Pasig City.

B. RAG Pipeline

The pipeline begins with **document loading**, where LangChain's `DirectoryLoader` reads the three text files stored in the `./my_data/` directory. The documents are then processed using `RecursiveCharacterTextSplitter` with a chunk size of 500 characters and a 50-character overlap. The overlap helps retain some surrounding context when information falls near the boundary between two chunks. The three source documents produced 20 text chunks during execution.

Next, the chunks are transformed into numerical vector representations using the `all-MiniLM-L6-v2` **embedding model**. These embeddings allow the system to compare the semantic meaning of the user's question with the contents of the document chunks instead of relying only on exact keyword matches.

The embeddings are stored in **ChromaDB**, which serves as the vector database. The retriever uses a top-k value of three, meaning that the three most relevant chunks are selected for each query. These chunks are then supplied as context to the language model before an answer is generated. The original laboratory template specified Groq's

llama-3.1-8b-instant as the generation model. However, during the implementation of this activity, the model had been deprecated by Groq for the intended setup. The implementation therefore used Groq's recommended replacement, openai/gpt-oss-20b , so that the RAG pipeline could continue using a supported model. This substitution affected the language model used for generation but did not change the main RAG architecture required by the activity. Document loading, chunking, embeddings, ChromaDB retrieval, and prompt-based grounding were retained.

Finally, LangChain's `create_stuff_documents_chain()` combines the retrieved context with the system prompt and language model, while `create_retrieval_chain()` connects the retriever and generation components. From my experience in building the pipeline, this connection is an important part of RAG because the chatbot is not simply given a question. It is first provided with information retrieved from the selected domain.

The resulting pipeline can be summarized as:

Documents → Chunking → all-MiniLM-L6-v2 → ChromaDB → Top-3 Retrieval → System Prompt → Groq LLM → Answer

II. IMPLEMENTATION DETAILS

Table I summarizes the main parameters used in the implementation. The baseline settings were retained for normal testing, while the temperature and fallback instruction were intentionally changed during the hallucination stress-test.

TABLE I. RAG IMPLEMENTATION PARAMETERS

RAG	
Parameters	Configuration
Source documents	3
Generated chunks	20
Chunk size	500
Chunk overlap	50
Embedding model	all-MiniLM-L6-v2
Vector database	ChromaDB
Top-k retrieval	3
Language model	openai/gpt-oss-20b
Baseline temperature	0
Stress-test temperature	1
Baseline fallback	Enabled
Stress-test fallback	Removed

A. Package and Runtime Compatibility

During the environment setup, I encountered package dependency warnings after installing the required LangChain and supporting libraries in Google Colab. Since

Colab already includes many Python packages, installing specific versions for the RAG environment resulted in some version requirements differing from packages already available in the runtime.

Specific versions of langchain, langchain-community , langchain-groq , and langchain-huggingface were therefore used together with ChromaDB and the required document-processing packages. Although dependency conflict warnings appeared during installation, the packages required by the RAG pipeline were successfully imported and the succeeding code cells executed correctly. The three documents were loaded, embeddings were generated, ChromaDB retrieval operated successfully, and the chatbot produced responses during testing.

This was a useful part of the activity because it showed me that an installation warning does not always mean that the specific workflow has failed. At the same time, dependency warnings should not simply be ignored. The succeeding code cells and outputs still need to be checked to confirm that the required components are functioning correctly.

III. EXPERIMENTAL RESULTS AND HALLUCINATIONS ANALYSIS

The chatbot was tested using an in-domain flood question and an intentionally out-of-domain question about the Moon landing. The second question was used for both the baseline test and hallucination stress-test so that the behavior of the two configurations could be compared directly.

TABLE II. TEST QUERIES AND MODEL OUTPUTS

Test	RAG		
	Query	Expected Response	Actual model Output
In-Domain	“What to do during flooding?”	Flood guidance from dataset	Returned flood-related safety guidance
Baseline Out-of-Domain	“Who was the first person to walk on the Moon, and in what year?”	Fallback message	“I cannot answer based on the provided domain data.”
Stress-Test	Same Moon question	Observe grounding behavior	“The first person to walk on the Moon was Neil Armstrong, and he did so in 1969 (Apollo 11 mission).”

For the in-domain test, the chatbot was asked “**What to do during flooding?**” This information was directly represented in the dataset, which contains instructions such as avoiding flooded areas, securing the home when safe, and following authorities when evacuation is announced. The retrieved source chunks came from the flood and typhoon preparedness document, showing that the retriever was able to connect the question with relevant domain information.

For the **baseline out-of-domain test**, the temperature was set to 0 and the explicit fallback rule remained active. The chatbot was asked:

"Who was the first person to walk on the Moon, and in what year?"

Because this information was not present in the disaster-preparedness dataset, the chatbot returned the required response:

"I cannot answer based on the provided domain data."

The result showed that the baseline configuration remained grounded when the retrieved documents did not contain an answer.

For the **hallucination stress-test**, the temperature was increased from 0 to 1.0 and the explicit fallback instruction was removed. The exact same question was asked again. This time, the chatbot answered:

"The first person to walk on the Moon was Neil Armstrong, and he did so in 1969 (Apollo 11 mission)."

Although the response contains information that is generally considered factually correct, the important observation in this experiment is that the information was **not supported by the retrieved Pasig City disaster-preparedness documents**. The purpose of the test was to evaluate grounding in the supplied domain, not the model's general knowledge.

TABLE III. BASELINE VS HALLUCINATION STRESS TEST

Table Column Head		
Aspect	Baseline	Stress-Test
Temperature	0	1
Fallback rule	Enabled	Removed
Answer in dataset	No	No
Behavior	Refused	Answered
Grounding	Maintained	Weakened
Unsupported information	No	Yes

The comparison helped me understand that a RAG hallucination should not be judged only by whether a statement is factually wrong. A response may be correct in general but still be unsupported by the retrieved context. It is also important not to attribute the result entirely to temperature. The experiment changed both the temperature and the fallback instruction, so the observed behavior reflects the modified configuration as a whole. In this test, the baseline provided stronger control over how the chatbot handled missing information.

IV. CONCLUSION

Building this RAG chatbot showed me that retrieval alone does not guarantee grounded responses. The retrieval stage successfully provides relevant document chunks, but prompt restrictions still influence whether the language model stays within those sources. The baseline experiment demonstrated this clearly: with an explicit fallback rule, the chatbot refused an unsupported question, while the modified configuration answered the same question using information outside the retrieved context.

Working with information from several disaster-preparedness sources also showed the importance of organizing domain data before retrieval. Bringing together Pasig City emergency information with relevant BFP-Pasig and DOST-PHIVOLCS guidance provided different types of information while keeping the dataset focused on disaster preparedness.

I also learned that reliability involves more than model responses. Package compatibility must be checked during implementation, and credentials should be handled carefully. Using getpass and an environment variable prevented the Groq API key from being displayed directly in the notebook. Overall, this activity made the relationship between **retrieval, prompt restrictions, and hallucination mitigation** much clearer to me. A convincing answer is not automatically a grounded answer. For a domain-specific RAG chatbot, the response should be supported by the information that the system was actually designed to retrieve.

REFERENCES

- [1] ITCC508, "Building and Evaluating a Domain-Specific RAG Chatbot," *Laboratory Exercise PT-M1, Introduction to LLMOps and RAG Concepts*, 2026.
- [2] Pasig City Government, "Tingnan: Pagbibigay ng fire safety tips mula sa Bureau of Fire Protection," *Pasig City Government*. [Online]. Available: [Pasig City Government – Fire Safety Tips](#)
- [3] Pasig City Government, "Flood safety tips before, during, and after the flood," *Pasig City Government*. [Online]. Available: [Pasig City Government – Flood Safety Tips](#)
- [4] Pasig City Public Information Office, "Emergency go bag preparedness information," *Facebook*. [Online]. Available: [Pasig City – Emergency Go Bag Post](#)
- [5] Pasig City Public Information Office, "Evacuation update," *Facebook*, Aug. 9, 2026. [Online]. Available: [Pasig City PIO – Evacuation Update](#)
- [6] Pasig City Government, "Emergency contact numbers ng Lungsod ng Pasig," *Pasig City Government*. [Online]. Available: [Pasig City Government – Emergency Contact Numbers](#)